

Structured Reasoning in LLM Optimization Agents: Scaffolding, Not Regularization

Kartik G Bhat*

Abstract

LLM-based optimization agents increasingly produce structured reasoning artifacts—hypothesis summaries, causal models, prediction logs—that persist across iterations. The assumption is that forcing articulation regularizes reasoning, as the self-explanation effect suggests it does for human learners. We test this assumption using SynthOracle, a family of synthetic multi-objective optimization oracles with known causal structure that enables separate measurement of optimization quality and reasoning quality. We pair the benchmark with a verbal regularization (VR) protocol that requires the agent to hypothesize, predict, and reconcile at each iteration via a structured summary embedded in conversation context. Through controlled ablations on two oracles and two models (Claude Opus 4.6 and Sonnet 4.6; $n = 5$ paired seeds per condition), we find that the forced summary is *scaffolding*, not regularization: it improves optimization for a less capable model (Sonnet, +26%) and a harder task (12-dimensional with noise, +14% to +35%), but *hurts* the most capable model on the simplest oracle (Opus, −35%, $p = 0.0006$). The pattern is consistent across 18 of 20 paired seeds (binomial $p = 0.0002$). A mechanism-isolation experiment shows that the performance penalty comes from the summary’s *persistence* in conversation context, not from the cognitive cost of producing it: an agent that produces the summary but does not receive it back performs identically to one that never produces it ($p = 0.35$). We connect this finding to the architectural trend in frontier reasoning models toward ephemeral chain-of-thought traces and derive a design principle: match the rigidity of your reasoning protocol to the gap between model capability and task difficulty.

1 Introduction

Large language models (LLMs) are increasingly deployed as autonomous agents for scientific optimization—selecting experiments, forming hypotheses, and iterating toward optimal designs. Recent systems pair LLMs with Bayesian optimization (BO) [1], embed them in closed-loop experimental workflows [2], or use them as surrogate-guided searchers [3]. A common design choice across these systems [1, 2] is to require the agent to produce structured reasoning artifacts: chain-of-thought traces, hypothesis summaries, or causal model descriptions that persist across iterations.

The assumption behind this choice borrows from human cognition. The self-explanation effect [4] demonstrates that articulating understanding improves learning: students who explain material to themselves acquire deeper knowledge than those who do not. For human scientists, writing down a model forces coherence and exposes gaps—a form of cognitive regularization.

Yet rigorous evidence for this assumption in LLM agents is thin. Gupta et al. [5] showed that BioDiscoveryAgent, despite producing “Reflection” and “Research Plan” text at each iteration, exhibits zero sensitivity

*Independent Researcher. Contact: gkartik@gmail.com. Code and data: <https://github.com/kar-ganap/synthoracle>.

to whether experimental feedback is real or permuted—suggesting the reasoning text is decorative. BORA [1] demonstrated that LLM-generated hypotheses improve Bayesian optimization, but the hypotheses are correlational, not causal, and the system does not verify whether the LLM actually updates its beliefs from data. Neither system can measure reasoning quality independently of optimization quality, because neither has access to ground-truth causal structure.

This measurement gap is the starting point for our work. Without ground truth, one cannot distinguish an agent that optimizes well because it understood the system from one that optimizes well despite misunderstanding it. To close this gap, we introduce **SynthOracle**, a family of synthetic multi-objective optimization oracles with known causal structure. Each oracle defines a directed acyclic graph (DAG) from inputs through mechanisms to outputs, with functional forms composed of standard building blocks (sigmoid, exponential, power law) in non-standard combinations designed to be unlikely to appear in LLM training data. The benchmark enables separate measurement of optimization quality (hypervolume relative to Bayesian optimization) and reasoning quality (precision and recall of discovered causal edges against the ground-truth DAG).

We pair the benchmark with a *verbal regularization* (VR) protocol: the agent iteratively hypothesizes causal relationships, makes quantitative predictions, reconciles prediction errors with observations, and produces a structured iteration summary—a schema-parsed object specifying claimed edges, confidence scores, surprises, and next-iteration plans. This summary is embedded in the condensed conversation context that persists across iterations, serving as the agent’s articulated causal model.

Our central finding contradicts the regularization assumption. Through a series of controlled ablations on two oracles (BASELINE: 6 inputs, all relevant; NOISY: 12 inputs, 6 irrelevant) and two models (Opus, Sonnet), we show:

1. **The forced summary is scaffolding, not regularization.** It helps when the agent needs structure—a less capable model (Sonnet) or a harder task (Noisy, where screening irrelevant dimensions benefits from articulated beliefs). It hurts when the agent does not need it—the most capable model (Opus) on the simplest oracle (BASELINE), where it reduces optimization quality by 35% (paired t -test, $p = 0.0006$, $n = 5$). Across all four model–oracle combinations, 18 of 20 paired seeds follow the predicted direction (binomial $p = 0.0002$).
2. **The mechanism is context cementing, not cognitive cost.** An agent that produces the structured summary but does not receive it back in subsequent context performs identically to one that never produces it (paired t -test, $p = 0.35$, $n = 5$). Both outperform the full VR agent ($p = 0.002$). The act of articulation is free; the persistence of articulated beliefs in conversation context cements early—often wrong—causal claims into subsequent reasoning.
3. **The agent genuinely learns from data.** A permuted-feedback ablation shows 66% higher optimization quality with real versus scrambled feedback, contradicting the finding of Gupta et al. [5] in the continuous-structured setting. Per-iteration surprise counts decrease monotonically across both oracles ($n = 5$, 144 evaluations).

These findings yield a design principle for practitioners building LLM optimization agents: match the rigidity of your reasoning protocol to the gap between your model’s capability and your task’s difficulty. The context-cementing mechanism also connects to a broader architectural pattern in frontier reasoning models, where chain-of-thought traces are produced but kept ephemeral rather than persisted in conversation context—a design choice consistent with the mechanism we identify.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 introduces the SynthOracle benchmark. Section 4 describes the verbal regularization protocol. Section 5 presents experimental results. Section 6 discusses implications and limitations. Section 7 concludes.

2 Related Work

Our work intersects three lines of research: LLM-assisted optimization, feedback sensitivity in LLM agents, and interpretable intermediate representations.

2.1 LLM-assisted optimization

Several recent systems integrate LLMs into optimization loops. BORA [1] defines an adaptive three-action policy: vanilla Bayesian optimization when the Gaussian process (GP) is confident, LLM-generated candidates when it is uncertain, and a collaborative mode at intermediate uncertainty. The system outperforms five baselines on mixed synthetic and chemistry benchmarks, demonstrating that LLM priors accelerate early-stage search. However, BORA’s LLM hypotheses are correlational (“increasing temperature improves yield”), not mechanistic, and the system does not track whether the LLM updates its beliefs from experimental outcomes. Boiko et al. [2] deploy an LLM agent for autonomous chemical synthesis with tool access, demonstrating multi-step planning, but without controlled comparison to non-LLM baselines or measurement of reasoning quality.

Our work differs in two respects. First, we require the agent to articulate *causal* hypotheses—directed edges with confidence scores—rather than correlational statements, enabling measurement of reasoning quality via edge precision and recall against ground truth. Second, we ablate the reasoning protocol itself, isolating whether structured articulation helps or hurts, rather than comparing LLM-assisted optimization to LLM-free optimization.

2.2 Feedback sensitivity in LLM agents

Gupta et al. [5] introduced a permuted-feedback ablation for LLM experimental-design agents: scramble the mapping between candidate selections and observed outcomes while preserving marginal distributions. Applied to BioDiscoveryAgent on discrete library-selection tasks, the ablation revealed zero feedback sensitivity—the agent performs equally well with real and fake outcomes. The authors conclude that LLM value derives entirely from priors (domain knowledge encoded during pre-training), not from posterior updating on experimental data. Their proposed alternative, LLMNN (LLM Nearest Neighbor), separates the LLM’s role into seed-point generation (priors only) and nearest-neighbor expansion (classical methods).

Our setting differs in three structural ways that may explain the divergent result. First, our oracle is continuous (6–12 real-valued inputs) rather than discrete (library selection), providing richer gradient information. Second, our protocol requires *quantitative predictions* before each evaluation, creating an explicit prediction-error signal the agent must reconcile. Third, our oracle has mechanistic structure (interacting mechanisms, thresholds, regime transitions) that rewards systematic exploration over random sampling. Under these conditions, we observe strong feedback sensitivity: 66% higher hypervolume with real versus permuted feedback.

2.3 Concept bottleneck models

Concept bottleneck models (CBMs) [6] force a neural network to route predictions through a layer of human-interpretable concepts before producing outputs. When the concepts are correct and sufficient, the bottleneck regularizes the model by preventing reliance on spurious features. When concepts are wrong

or incomplete, the bottleneck constrains the model to an incorrect intermediate representation, degrading performance [7].

Our structured iteration summary functions as a *learned* concept bottleneck: the agent constructs its own interpretable intermediate representation (claimed causal edges with confidence scores) that mediates between raw observations and optimization decisions. The key difference from standard CBMs is that the bottleneck is not designed by the researcher but discovered by the agent—and its quality depends on the agent’s capability and the task’s difficulty. Our 2×2 matrix (Section 5) empirically confirms the CBM prediction: the bottleneck helps when concepts are correct (screening noise dimensions on **NOISY**) and hurts when early concepts are wrong (threshold effects on **BASELINE**).

2.4 Chain-of-thought and structured reasoning

Chain-of-thought prompting [8] and ReAct [9] demonstrate that eliciting intermediate reasoning steps improves LLM performance on single-turn tasks. Our work examines a different question: what happens when structured reasoning artifacts *persist across turns* in a multi-iteration agent loop? The distinction matters because persistence creates a commitment device—the agent’s prior reasoning anchors subsequent iterations—which can be beneficial (maintaining useful beliefs) or harmful (cementing wrong ones). This is orthogonal to the single-turn finding that reasoning steps help: producing a chain of thought within one iteration may be universally beneficial, but carrying it forward into the next iteration’s context is not.

3 SynthOracle Benchmark

SynthOracle is a family of synthetic multi-objective optimization oracles designed to evaluate whether LLM agents learn causal structure, not just find good solutions. Each oracle is a deterministic Python function $f : \mathbb{R}^d \rightarrow \mathbb{R}^4$ mapping d inputs to four outputs with mixed optimization directions (maximize Y_1 , minimize Y_2 , threshold $Y_3 > 0.4$, maximize Y_4). Evaluations take <1 ms and require no API calls, enabling high-throughput experimentation at zero marginal cost.

3.1 Oracle family

The benchmark comprises four oracles that share a common mechanism topology but vary along independent structural axes (Figure 1, Table 1). All four route inputs through four intermediate mechanisms (M_1 – M_4 : throughput, leakage, coupling, and threshold gating) before producing outputs. The functional forms combine standard building blocks—sigmoid activations, exponential decay, power laws, rational functions—in non-standard compositions unlikely to appear in LLM training corpora. Full equations and parameter values are provided in Section A.

BASELINE The reference oracle. Six inputs, all causally relevant, with 26 mechanism-level edges (18 when projected to input–output pairs). Features include a regime transition at $X_1 \approx 0.27$, a hidden coupling intermediate ($M_3 = X_4/(X_4 + X_6)$) that modulates two mechanisms, and a sharp threshold activation at $X_5 = 0.38$.

SHIFTED Same mechanism topology as **BASELINE**, but with altered functional forms: the threshold is disabled (gate $\equiv 1$), a monotonic relationship becomes inverted-U, and an output sign is flipped. Tests whether an agent can *unlearn* incorrect functional-form assumptions when the causal graph is preserved.

REWired Same number of mechanisms but with input–mechanism wiring swapped: the inputs that drove throughput (M_1) in **BASELINE** now drive leakage (M_2) and vice versa. Tests whether an agent can *dismiss*

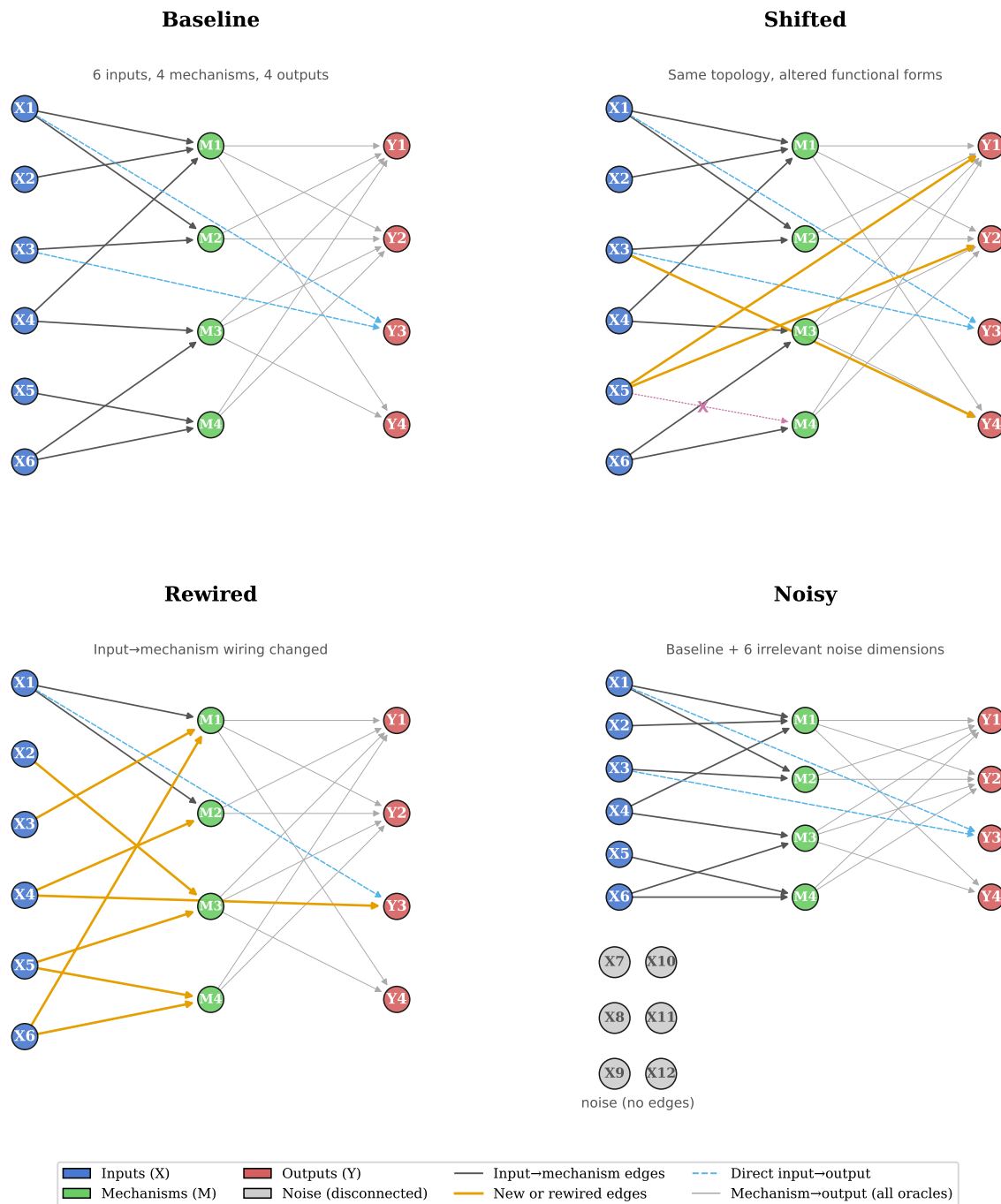


Figure 1. Causal structure of the four SynthOracle oracles. Inputs (blue) connect to mechanisms (green) which produce outputs (red). Dark arrows: shared edges. Orange arrows: new or rewired edges relative to BASELINE. Dashed blue: direct input→output paths. Gray nodes in Noisy: disconnected noise dimensions. Mechanism→output edges (light gray) are shared across all oracles.

Table 1. Oracle intrinsic properties. d : input dimensionality. k : number of mechanisms. $R^2(M \rightarrow Y)$: fraction of output variance explained by mechanism activations (mean across outputs). IO edges: number of input–output (IO) edges in the projected ground-truth DAG.

Oracle	d	k	k/d	$R^2(M \rightarrow Y)$	IO edges	Noise dims	What varies vs. BASELINE
BASELINE	6	7	1.17	0.961	18	0	Reference
SHIFTED	6	7	1.17	0.748	19	0	Functional forms
REWired	6	7	1.17	0.960	19	0	Input–mechanism wiring
NOISY	12	7	0.58	0.961	18	6	6 irrelevant dimensions

prior structural beliefs when the topology changes. Transfer experiments using SHIFTED and REWired are reported in Section E.

NOISY Identical to BASELINE but embedded in a 12-dimensional input space: the six real inputs (X_1 – X_6) retain their causal effects; six noise inputs (X_7 – X_{12}) have zero effect on all outputs. Tests the agent’s ability to *screen* irrelevant dimensions. The mechanism-to-input ratio drops from $k/d = 1.17$ (BASELINE) to $k/d = 0.58$: an agent that identifies the 6 noise inputs via causal reasoning effectively reduces its search space from 12 to 6 dimensions, while BO’s Gaussian process must model all 12.

3.2 Diagnostic suite

The key property of SynthOracle is that each oracle’s ground-truth DAG is known, enabling measurement of reasoning quality independently of optimization quality. We define four diagnostic metrics.

Edge precision and recall. The ground-truth DAG contains mechanism-level edges (e.g., $X_1 \rightarrow M_1 \rightarrow Y_1$). The IO projection collapses these by tracing reachability: if any path exists from X_i to Y_j through mechanisms, the edge $X_i \rightarrow Y_j$ is included. The agent’s discovered edges are compared against this projected DAG. An edge is counted as discovered if, when replaying the agent’s one-at-a-time (OAT) sweeps on the oracle, the output range exceeds a threshold of 0.05 (5% of output scale). Precision measures the fraction of claimed edges that are real; recall measures the fraction of real edges that are found. Sensitivity analysis confirms that precision is 1.00 for all thresholds tested (0.01–0.20) due to complete separation between true-edge and noise-edge effect sizes (Section C).

Screening efficiency. On NOISY, we measure the fraction of OAT sweeps directed at noise inputs and the maximum confidence assigned to any noise edge in the agent’s final iteration summary. An agent with perfect screening spends zero evaluation budget on noise dimensions and assigns zero confidence to noise edges.

Calibration learning. At fixed evaluation counts, the agent is asked to predict outputs for a held-out test point. The mean absolute error (MAE) between predicted and actual outputs measures prediction quality. A decrease in MAE from the first to last calibration checkpoint indicates the agent’s internal model is improving.

Surprise trajectory. At each iteration, the agent reports events that contradicted its current model. A decreasing surprise count indicates the agent is updating its beliefs to match the data.

The benchmark additionally provides Sobol-weighted information capture, OAT prediction accuracy, sample efficiency curves, and adversarial region error; these are reported in Section D.

3.3 Bayesian optimization baseline

We compare VR against multi-objective Bayesian optimization using qNEHVI [10] with a standard Gaussian process surrogate, implemented in BoTorch. Both methods receive the same Latin Hypercube Sampling (LHS) initialization ($2d$ points). BO runs use batch size 1 with sequential acquisition. We report hypervolume (HV) as the primary optimization metric, computed against a reference point derived from the oracle’s output ranges.

4 Verbal Regularization Protocol

The VR agent is an LLM-driven optimization loop that interleaves oracle evaluations with structured causal reasoning. The agent receives a system prompt describing the oracle’s inputs, outputs, optimization directions, and the feasibility constraint $Y_3 > 0.4$ (designs violating this constraint are infeasible), but no information about the underlying mechanism structure. It iterates until the evaluation budget is exhausted.

4.1 Tool suite

The agent interacts with the oracle through two categories of tools.

Evaluation tools consume budget (one oracle call each):

- *Point evaluation*: evaluate the oracle at a specific input vector; returns outputs and prediction errors (if the agent provided predictions).
- *OAT sweep*: one-at-a-time sensitivity sweep—vary one input across its range while holding others fixed; returns output trajectories and predicted-vs-actual trend comparisons.
- *Interaction test*: $2 \times n$ factorial design for two inputs; returns interaction effect sizes.

Analysis tools are free (computed from existing data): pairwise correlation matrix, input sensitivity estimates, local finite-difference gradients, current Pareto front with hypervolume, and linear regression fits. These allow the agent to analyze accumulated data without spending evaluation budget.

The tool design embodies the VR principle: evaluation tools require the agent to make *quantitative predictions* before observing results. For point evaluations, the agent predicts output values; for OAT sweeps, it predicts monotonic trend directions. Prediction errors are returned alongside actual values, creating an explicit reconciliation signal.

4.2 Iteration loop

Each iteration proceeds as follows:

1. The agent receives a condensed context containing: the current evaluation count and budget remaining, output value ranges observed so far, the most recent evaluation results, and (if summary generation is enabled) the agent’s prior structured summary under a heading “Your Causal Model.”
2. The agent makes up to 15 tool calls per iteration, freely choosing which tools to invoke and in what order.
3. After tool calls complete, the agent produces a *structured iteration summary*: a structured object parsed against a fixed schema, containing:
 - *Edges*: a list of claimed causal edges, each with a source, target, confidence score (0–1), and brief rationale;
 - *Surprises*: observations that contradicted the agent’s current model;
 - *Findings*: mechanistic insights gained this iteration;

- *Plan*: what the agent intends to investigate next.
4. At configurable intervals (every 20 evaluations by default), a *calibration checkpoint* asks the agent to predict outputs for a random held-out input, measuring prediction quality over time.

Between iterations, the conversation is condensed: the full tool-call history is replaced with a compact status message, preserving the structured summary as persistent context. This condensation is necessary because multi-iteration runs with 15 tool calls per iteration would otherwise exceed the model’s context window. It is also the mechanism through which the structured summary *cements* into subsequent reasoning—the ablation target in our experiments.

4.3 Ablation parameters

The protocol exposes two boolean parameters that enable the controlled ablations reported in Section 5:

- *Skip summary*: the agent does not produce the structured summary. It retains access to all tools and makes the same number of tool calls, but no iteration summary is generated or persisted.
- *Summary not in context*: the agent produces the structured summary (which is logged for analysis), but the summary is *not* embedded in the condensed context for the next iteration. The agent receives a generic status message instead of its prior causal model.

These two parameters create a clean three-condition experimental design: (1) summary produced and in context (full VR), (2) summary produced but not in context, and (3) no summary produced. Comparing conditions 1 vs. 2 isolates the effect of *persistence*; comparing 2 vs. 3 isolates the effect of *articulation*.

4.4 Implementation

All experiments use the Anthropic Messages API with adaptive extended thinking (Section B provides the full prompt template and output schema). The default model is Claude Opus 4.6; model-comparison experiments additionally use Sonnet 4.6 and Haiku 4.5. Maximum output tokens are set to 16,000 (24,000 for 144-evaluation extended runs). Latin Hypercube Sampling provides $2d$ initial points (12 for BASELINE, 24 for Noisy) shared across VR and BO conditions for each seed.

5 Experiments

All VR experiments use $n = 5$ seeds (42–46) and all BO experiments use $n = 3$ seeds (42–44) unless otherwise noted. BO’s lower seed count reflects its lower variance: the deterministic acquisition function (qNEHVI) introduces variability only through the LHS initialization.

5.1 VR pays an understanding tax that scales with k/d

Table 2 summarizes optimization and causal discovery results; Figure 2 shows the HV trajectories on BASELINE and Noisy at 144 evaluations. On BASELINE, VR reaches 69% of BO at 72 evaluations and converges to 99% at 144. On Noisy, VR reaches 94% at 72 evaluations and 98% at 144. The gap between the two oracles reflects the mechanism-to-input ratio k/d : when irrelevant dimensions outnumber mechanisms (Noisy, $k/d = 0.58$), VR’s ability to screen inputs via causal reasoning provides a structural advantage that partially offsets the understanding tax.

Edge precision is 1.00 across all conditions—the agent never claims a false causal edge. Recall is 0.94 on BASELINE (the consistently missed edge is the hidden threshold $X_5 \rightarrow Y_2$) and rises from 0.70 at 72 evaluations to 0.94 at 144 on Noisy, indicating that extended budget enables more complete causal discovery.

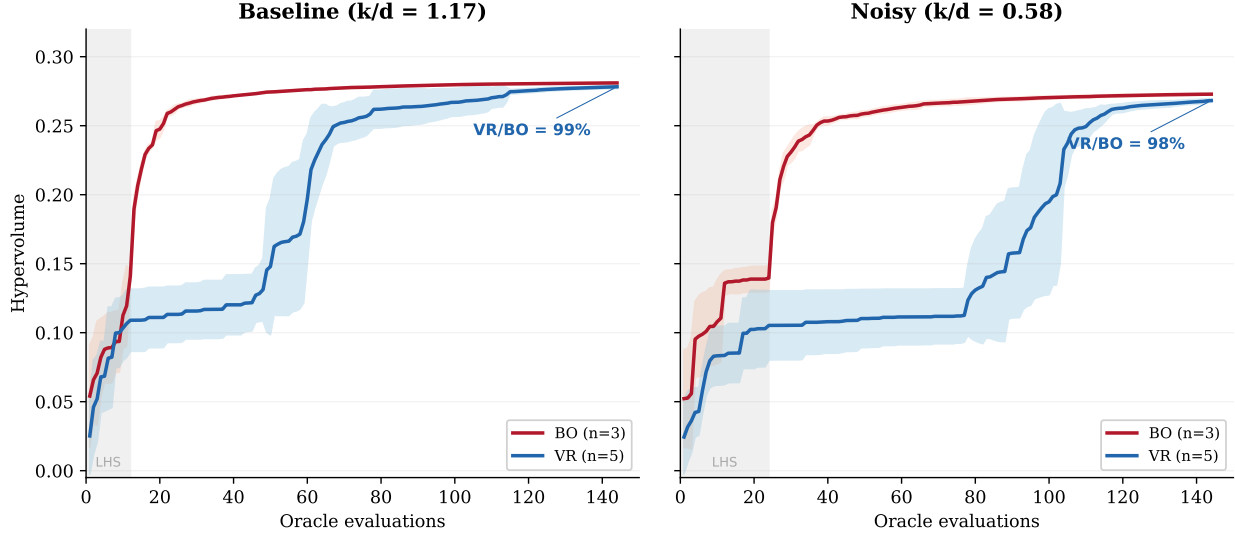


Figure 2. Hypervolume trajectories on BASELINE and NOISY (144 evaluations). VR: $n = 5$ seeds (mean $\pm$ 1 s.d. shaded). BO: $n = 3$ seeds. Shaded region at left: Latin Hypercube Sampling initialization. VR converges to 99% of BO on BASELINE and 98% on NOISY. On NOISY, the agent invests the first ~ 80 evaluations in model-building (flat HV) before exploiting.

Table 2. Optimization and causal discovery results. VR: $n = 5$ seeds. BO: $n = 3$ seeds. Edge precision and recall computed by replaying OAT sweeps against the IO-projected ground-truth DAG (threshold = 0.05).

Oracle	Budget	VR HV	BO HV	VR/BO	Precision	Recall
BASELINE	72	.192 $\pm$.015	.278 $\pm$.001	69%	1.00	0.94
	144	.278 $\pm$.001	.281 $\pm$.000	99%	1.00	0.94
NOISY	72	.251 $\pm$.017	.267 $\pm$.001	94%	1.00	0.70
	144	.268 $\pm$.001	.273 $\pm$.001	98%	1.00	0.94

5.2 The agent uses feedback, contra Gupta et al.

We replicate the permuted-feedback ablation of Gupta et al. [5] on BASELINE. Two VR runs share the same LHS initialization (seed 42, 12 points): one receives real oracle outputs; the other receives outputs with the candidate–outcome mapping scrambled (preserving marginal distributions). Figure 3 shows the result: trajectories overlap during LHS and diverge immediately afterward (evaluation 13). The real-feedback agent reaches HV = 0.219; the permuted agent plateaus at 0.132—a 66% difference.

This contradicts the finding of Gupta et al. that LLM agents show zero feedback sensitivity on discrete experimental-design tasks. In our continuous-structured setting, where the agent makes quantitative predictions and receives explicit prediction errors, feedback is load-bearing.

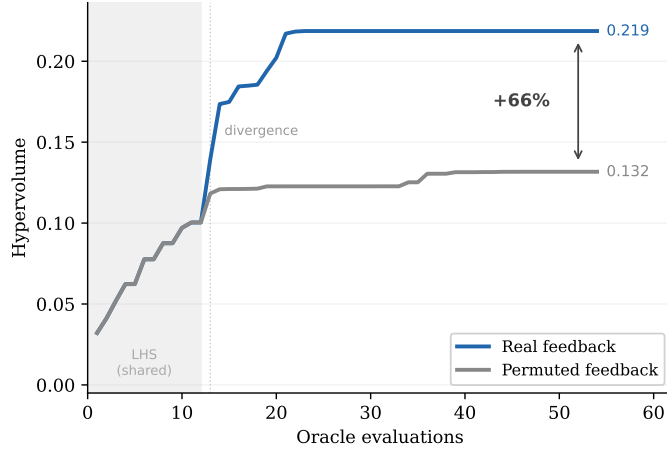


Figure 3. Permuted-feedback ablation on BASELINE (seed 42). Both conditions share the same LHS initialization (12 points). Trajectories diverge at evaluation 13. Real feedback: HV = 0.219. Permuted: 0.132 (+66% effect).

5.3 The agent’s causal model improves over iterations

Figure 4 shows per-iteration surprise counts for both oracles at 144 evaluations ($n = 5$ matched seeds). Surprise counts decrease monotonically from ~ 5 at iteration 1 to ~ 2 at iteration 4 on both BASELINE and NOISY, indicating the agent systematically updates its causal model to better match the data. The decline is consistent across seeds (error bars overlap between oracles), suggesting the learning dynamic is robust to oracle structure. Surprise counts do not reach zero—the agent’s causal model remains imperfect (recall = 0.94, not 1.00), and the consistently missed edge ($X_5 \rightarrow Y_2$) continues to generate prediction errors.

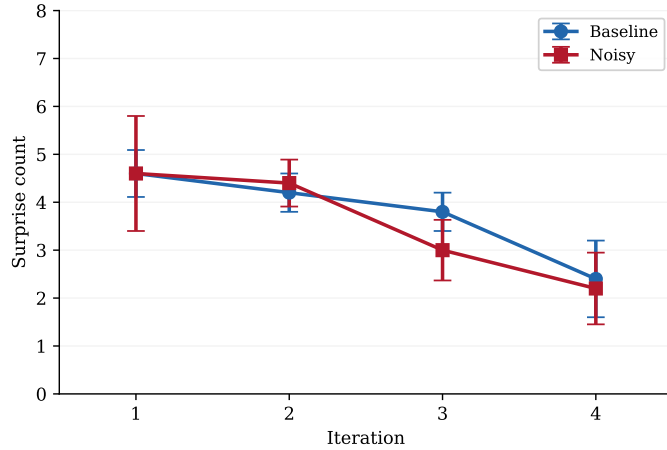


Figure 4. Per-iteration surprise count on BASELINE and NOISY (144 evaluations, $n = 5$ matched seeds 42–46). Both oracles show monotonic decline, indicating the agent updates its causal model from data. Truncated to 4 iterations (the minimum across all seeds in both conditions). Error bars: ± 1 s.d.

5.4 Forced summaries are scaffolding, not regularization

We ablate the structured iteration summary across two models (Opus 4.6, Sonnet 4.6) and two oracles (BASELINE, NOISY), creating a 2×2 matrix of paired comparisons ($n = 5$ seeds per cell). For each seed, the “with summary” and “without summary” conditions share the same LHS initialization, isolating the summary’s effect.

Table 3 and Figure 5 present the results. The summary hurts in exactly one cell: Opus on BASELINE (−35%,

Table 3. Summary ablation: 2×2 matrix. $n = 5$ paired seeds per cell (seeds 42–46). Effect: percentage change in mean HV due to adding the summary. p -values from paired t -tests.

Cell	With summary	Without	Effect	p	Dir.
Opus $\times$ BASELINE	.192 $\pm$.015	.259 $\pm$.014	−35%	0.0006	5/5
Sonnet $\times$ BASELINE	.258 $\pm$.013	.204 $\pm$.041	+26%	0.054	5/5
Opus $\times$ NOISY	.251 $\pm$.017	.219 $\pm$.042	+14%	0.095	4/5
Sonnet $\times$ NOISY	.243 $\pm$.020	.202 $\pm$.034	+21%	0.169	4/5
Global sign consistency: 18/20 seeds				$p = 0.0002$	

$p = 0.0006$). In the other three cells, the summary helps: Sonnet on BASELINE (+26%, $p = 0.054$), Opus on NOISY (+14%, $p = 0.095$), and Sonnet on NOISY (+21%, $p = 0.169$). While individual p -values for the $n = 5$ cells vary, 18 of 20 paired seeds across all four cells follow the predicted direction (binomial $p = 0.0002$).

The pattern admits a clean interpretation: the summary is *scaffolding*, not regularization. It helps when the agent needs structure—a less capable model (Sonnet benefits on both oracles) or a harder task (both models benefit on NOISY, where screening noise dimensions requires systematic tracking of which inputs matter). It hurts only in the one cell where the agent does not need structure: the most capable model (Opus) on the simplest oracle (BASELINE, where all inputs are relevant and no screening is needed).

Without the summary, raw tool-use capability reveals a 35% model gap on BASELINE: Opus reaches 0.259, Sonnet only 0.204. The summary lifts Sonnet to 0.258—near parity with unscaffolded Opus—but drags Opus down to 0.192. The summary equalizes performance by helping the weaker model and constraining the stronger one.

This finding also clarifies the “understanding tax” reported in Section 5.1. On BASELINE at 72 evaluations, VR with the summary reaches 69% of BO, while VR without the summary reaches 92%. The total 31-percentage-point gap between VR-with-summary and BO decomposes as: $100\% - 92\% = 8$ percentage points from VR’s exploration overhead (OAT sweeps, interaction tests versus BO’s GP-based acquisition), and $92\% - 69\% = 23$ percentage points from the summary cementing early beliefs. On BASELINE, the summary accounts for roughly three-quarters of the understanding tax. On NOISY, where the summary aids screening, both components shrink and VR approaches BO parity. The understanding tax and the scaffolding finding address different questions: the former asks whether VR is worth using at all (the answer depends on whether causal understanding is valued alongside optimization); the latter asks how to configure VR once adopted (match summary rigidity to the model–task capability gap).

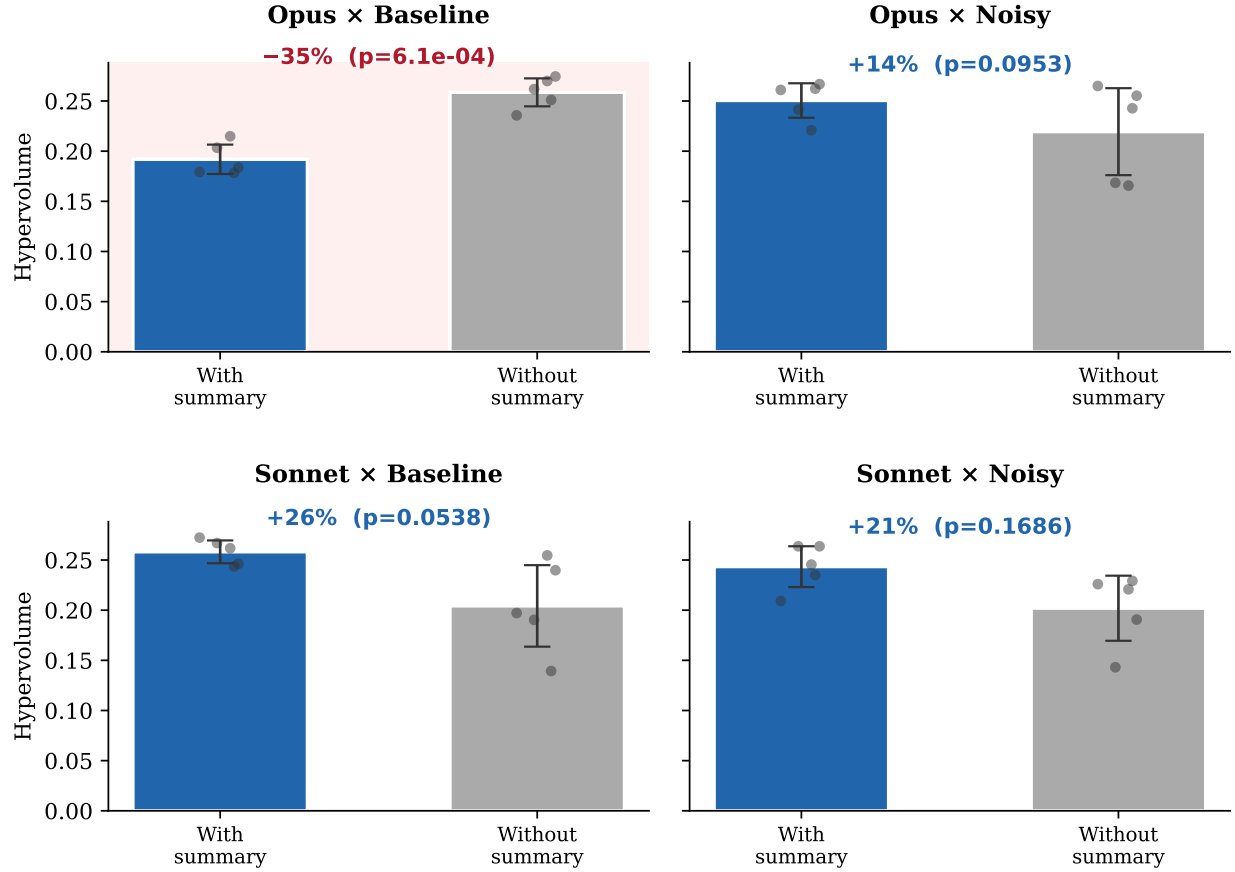


Figure 5. Summary ablation: 2×2 matrix. Each panel shows mean HV with and without the structured summary ($n = 5$ paired seeds). Error bars: ± 1 s.d. Dots: individual seeds. Pink background: the one cell where the summary hurts. The summary helps in 3 of 4 cells; it hurts only for the most capable model on the simplest oracle.

5.5 The penalty is from persistence, not articulation

We run a third condition on BASELINE with Opus ($n = 5$, seeds 42–46): the agent produces the structured summary (the schema parse occurs and the summary is logged), but the summary is *not* embedded in the condensed context for the next iteration. The agent receives a generic status message instead of “Your Causal Model.”

Figure 6 shows the result. The three conditions are:

- Full VR (summary produced and in context): 0.192 ± 0.015
- Summary not fed back (produced but not in context): 0.268 ± 0.009
- No summary (never produced): 0.259 ± 0.014

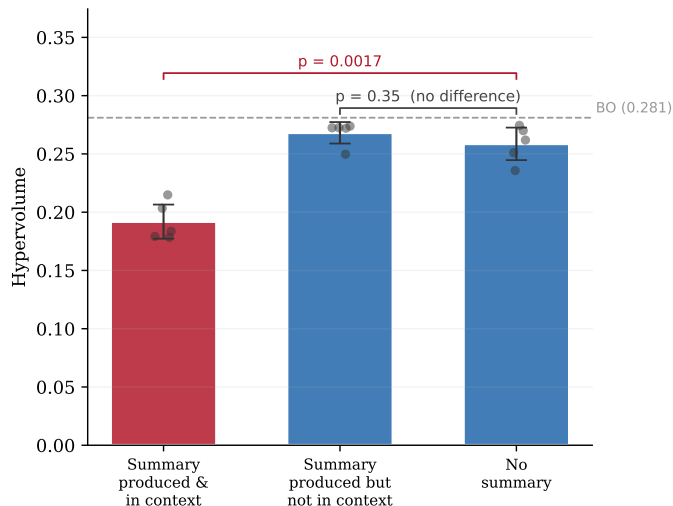


Figure 6. Mechanism isolation on BASELINE (Opus, $n = 5$, seeds 42–46). The summary-not-fed condition (produced but not persisted in context) performs identically to no summary ($p = 0.35$). Both significantly outperform full VR ($p = 0.002$). Dashed line: BO reference. The act of articulation is free; persistence is costly.

The summary-not-fed condition performs identically to the no-summary condition (paired t -test, $p = 0.35$) and significantly outperforms the full VR condition ($p = 0.002$). The act of producing the structured summary—including the LLM call, the token cost, and the cognitive work of articulation—has zero measurable effect on optimization quality. The entire performance gap comes from *persisting* the summary in the conversation context, where it cements early beliefs (including incorrect ones such as dismissing $X_5 \rightarrow Y_2$ at confidence 0.1) into subsequent iterations.

5.6 Discovery decouples from exploitation below Sonnet tier

We run three models on Noisy at 72 evaluations ($n = 3$ seeds each): Opus 4.6, Sonnet 4.6, and Haiku 4.5. Figure 7 shows the results.

Opus and Sonnet achieve near-parity with BO (97% and 96% respectively) and maintain perfect edge precision—zero false-positive noise edges, with the highest confidence assigned to any noise-input edge at ≤ 0.22 out of 1.0, indicating clear dismissal. The two models achieve similar outcomes via different cognitive strategies: Opus dismisses noise inputs by inference alone (0% of OAT sweeps on noise dimensions), while Sonnet verifies empirically (20% noise OAT sweeps) before dismissing.

At Haiku tier, a qualitative break occurs. HV drops to 44% of BO, and the agent claims two noise edges at confidence 0.55 and 0.60—the only false positives in the entire experimental suite. Yet Haiku’s Sobol-weighted

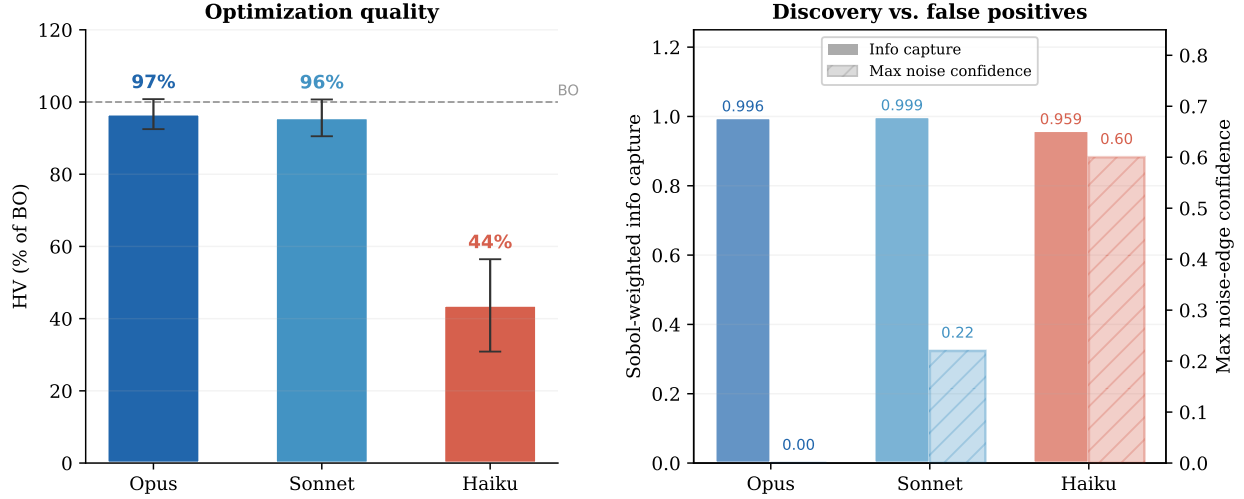


Figure 7. Model capability gradient on Noisy (72 evaluations, $n = 3$ seeds each). Left: HV as percentage of BO. Opus and Sonnet achieve near-parity ($\geq 96\%$); Haiku collapses to 44%. Right: Sobol-weighted information capture (solid bars) vs. maximum noise-edge confidence (hatched bars). Haiku discovers the correct causal structure (info capture 0.96) but cannot exploit it and generates false-positive noise edges.

information capture remains 0.96—near parity with Opus and Sonnet. Haiku *discovers* the correct causal structure but cannot *exploit* it for optimization and cannot reliably distinguish real from spurious edges. This discovery–exploitation decoupling suggests the two capabilities have different model-capability thresholds.

6 Discussion

6.1 The concept bottleneck interpretation

The structured iteration summary functions as a learned concept bottleneck (Section 2): an interpretable intermediate representation that mediates between observations and decisions. Our 2×2 matrix empirically confirms the prediction from the CBM literature: bottlenecks help when concepts are correct and hurt when they are wrong. On Noisy, the agent’s early belief that noise dimensions are irrelevant is correct, and persisting this belief via the summary provides useful scaffolding. On BASELINE, the agent’s early belief that X_5 has weak influence (dismissed at confidence 0.1) is wrong— X_5 drives a hidden threshold mechanism—and persisting this incorrect belief cements the error.

A key difference from standard CBMs is that our bottleneck is *discovered*, not designed. In a CBM, the researcher specifies which concepts to include; if they choose wrong, performance suffers. In our setting, the agent constructs its own concepts (causal edges with confidence scores), and the quality of this construction depends on the agent’s capability and the task’s difficulty. This makes the scaffolding–regularization distinction model-dependent: the same protocol is scaffolding for one model and an impediment for another.

6.2 Ephemeral versus persistent reasoning traces

Our finding that the *persistence* of reasoning traces—not their production—drives the performance penalty raises a natural question about multi-turn LLM agent design more broadly: should reasoning artifacts be carried forward across turns, or treated as ephemeral? Recent reasoning-oriented models architecturally separate reasoning traces from conversation context—producing extended chain-of-thought tokens that are not presented to the user and may not persist verbatim into subsequent turns. Our three-condition ablation provides one controlled data point: in our setting, ephemerality is strictly better than persistence when

the persisted content contains early-stage causal beliefs. Whether this extends to other forms of persistent reasoning (e.g., maintained plans, accumulated observations) is untested.

6.3 Design principles

Our results suggest three practical guidelines for practitioners building LLM optimization agents:

1. **Match protocol rigidity to the capability gap.** Use structured summaries when the model needs help organizing its reasoning (less capable models, higher-dimensional tasks). Omit them when the model can effectively reason without imposed structure.
2. **Prefer ephemeral over persistent reasoning traces.** If structured articulation is used for interpretability or logging, consider not feeding the summary back into the agent’s context. Our mechanism-isolation experiment shows this preserves all the benefit of articulation (the summary is still produced and logged) while avoiding context cementing.
3. **There is a model capability floor for reliable causal reasoning.** Below a sufficient capability threshold (between Sonnet 4.6 and Haiku 4.5 in our experiments), causal discovery decouples from exploitation: the agent identifies structure but cannot use it effectively and may generate false-positive claims.

6.4 Limitations

Synthetic oracles only. SynthOracle’s functional forms are deterministic, noise-free, and low-dimensional relative to real scientific systems. Whether the scaffolding finding transfers to stochastic, high-dimensional physical simulators remains untested.

Single reasoning protocol. We evaluate one VR protocol design. Alternative structured-reasoning approaches (BORA’s adaptive policy, ReAct-style interleaving, simple free-text reflection) were not compared. Our finding is about the effect of *persisting* structured summaries, not about the specific VR protocol being optimal.

Sample sizes. Most conditions use $n = 5$ seeds. Only one of four cells in the 2×2 matrix reaches individual significance at $p < 0.05$ (Opus $\times$ BASELINE, $p = 0.0006$); the others are directional but underpowered ($p = 0.054, 0.095, 0.169$). Our primary statistical claim rests on the global sign test: 18 of 20 paired seeds follow the scaffolding prediction (binomial $p = 0.0002$). This tests the *pattern*—one cell hurts, three help—rather than individual cell significance. Additional seeds would tighten per-cell confidence intervals but the directional pattern is already robust. We pre-registered predictions for four experimental conditions before running them; three of four were qualitatively wrong (Section F), underscoring the value of controlled ablation over intuition.

Articulation format. The scaffolding effect may depend on *what* is articulated. Our protocol requires causal edges with confidence scores—a specific, structured format that cements specific beliefs. A protocol that articulated optimization priorities, uncertainty estimates, or free-text reflections instead might show different cell-by-cell effects. The finding that persistence hurts is likely general; the specific interaction with model capability and task difficulty may be format-dependent.

7 Conclusion

We introduced SynthOracle, a benchmark with ground-truth causal structure that enables separate measurement of optimization quality and reasoning quality in LLM agents. Using this benchmark, we tested the assumption

that forcing LLM agents to articulate structured reasoning regularizes their behavior—and found it wrong in general.

The forced structured summary is scaffolding, not regularization. It helps agents that need structure (less capable models, harder tasks with irrelevant dimensions) and hurts the most capable model on the simplest task. The mechanism is context cementing: persisting the summary in conversation context anchors early beliefs into subsequent iterations. The act of articulation itself is free—an agent that produces the summary but does not receive it back performs identically to one that never produces it.

This finding has a direct practical implication: when building LLM optimization agents, match the rigidity of your reasoning protocol to the gap between your model’s capability and your task’s difficulty. And if you use structured summaries for interpretability, consider making them ephemeral rather than persistent—you keep the logging benefit without the cementing cost.

More broadly, the context-cementing mechanism suggests that how LLM agents *remember* their reasoning matters as much as whether they reason at all. The architectural trend toward ephemeral chain-of-thought in frontier reasoning models is consistent with this principle. Our benchmark provides a controlled setting for future work on reasoning persistence, belief revision, and the conditions under which structured articulation helps versus harms autonomous scientific agents.

References

- [1] K. Cisse, Y. Shi, and S. Zhe. BORA: A modular LLM-based adaptive Bayesian optimization research agent. In *Proc. IJCAI*, 2025.
- [2] D. A. Boiko, R. MacKnight, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- [3] M. Acosta Ramos, P. A. Ortega, et al. Bayesian optimization with LLM surrogates. *arXiv preprint*, 2024.
- [4] M. T. H. Chi, N. de Leeuw, M.-H. Chiu, and C. LaVancher. Eliciting self-explanations improves understanding. *Cognitive Science*, 18(3):439–477, 1994.
- [5] A. Gupta, Y. Du, A. Nair, P. Agrawal, and F. Sala. Are we there yet? A critical analysis of LLM agents in closed-loop experimental design. *arXiv:2503.15547*, 2025.
- [6] P. W. Koh, T. Nguyen, Y. S. Tang, S. Mussmann, E. Pierson, B. Kim, and P. Liang. Concept bottleneck models. In *Proc. ICML*, pp. 5338–5348, 2020.
- [7] M. Yüsekgonül, M. Wang, and J. Zou. Post-hoc concept bottleneck models. In *Proc. ICLR*, 2023.
- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in NeurIPS*, 2022.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. ICLR*, 2023.
- [10] S. Daulton, M. Balandat, and E. Bakshy. Parallel Bayesian optimization of multiple noisy objectives with expected hypervolume improvement. In *Advances in NeurIPS*, 2021.

A Oracle Specifications

All oracles map inputs $X_i \in [0.1, 1.0]$ to four outputs with directions: maximize Y_1 , minimize Y_2 , threshold $Y_3 > 0.4$, maximize Y_4 . We use $\sigma(x) = 1/(1 + e^{-x})$ throughout.

A.1 BASELINE (reference)

Mechanisms:

$$\begin{aligned} M_1 &= X_2^{1.37} \cdot X_4^{0.82} \cdot (1 - e^{-5X_1}) && \text{(throughput)} \\ M_2 &= 0.55 \cdot e^{-1.8X_3\sqrt{X_1}} && \text{(leakage)} \\ M_3 &= X_4/(X_4 + X_6) && \text{(coupling)} \\ g &= \sigma(25(X_5 - 0.38)) && \text{(threshold gate)} \end{aligned}$$

Effective mechanisms:

$$\begin{aligned} M_1^{\text{eff}} &= M_1 \cdot M_3 & M_2^{\text{eff}} &= M_2 \cdot (1 - 0.6M_3) \\ M_4^\times &= 1 + 0.4X_6 \cdot g & M_4^+ &= 0.5X_6 \cdot g \end{aligned}$$

Outputs:

$$\begin{aligned} Y_1 &= M_1^{\text{eff}} \cdot M_4^\times + M_4^+ - M_2^{\text{eff}} \\ Y_2 &= 0.85M_2^{\text{eff}} + 0.35M_1^{\text{eff}}/M_4^\times + 0.25(1 - M_3) \\ Y_3 &= \sigma(8.1(X_1 - 0.27)) \cdot X_3^{0.5} \\ Y_4 &= M_1^{\text{eff}}/(1 + 2M_1^{\text{eff}}) \end{aligned}$$

A.2 SHIFTED (functional forms altered)

Same mechanism topology as BASELINE with the following changes:

- Coupling weakened: $M_3 = X_4/(X_4 + 0.5X_6)$
- Threshold disabled: $g \equiv 1$
- Y_1 : adds smooth X_5 contribution ($0.3X_5^{0.8}$) and $X_1 \times X_5$ interaction ($0.8X_1X_5$)
- Y_2 : M_1^{eff} sign flipped (-0.20 instead of $+0.35$); adds X_5 penalty ($0.15X_5$)
- Y_3 : inverted-U in X_3 (peaks at $X_3 \approx 0.6$) instead of $X_3^{0.5}$
- Y_4 : adds log contribution ($0.12 \ln(1 + 3X_3)$); weaker saturation (1.5 instead of 2.0)

A.3 REWIRED (input-mechanism wiring swapped)

Same functional forms as BASELINE, but with input assignments permuted:

- M_1 : $X_3^{1.37} \cdot X_6^{0.82} \cdot (1 - e^{-5X_1})$ (was X_2, X_4)
- M_2 : $0.55 \cdot e^{-1.8X_4\sqrt{X_1}}$ (was X_3)
- M_3 : $X_2/(X_2 + X_5)$ (was X_4, X_6)
- Gate: $\sigma(25(X_6 - 0.38))$ (was X_5)
- M_4 : uses $X_5 \cdot g(X_6)$ (was $X_6 \cdot g(X_5)$)
- Y_3 : $\sigma(8.1(X_1 - 0.27)) \cdot X_4^{0.5}$ (was X_3)

A.4 Noisy (12-dimensional)

Identical to BASELINE for inputs X_1 – X_6 . Inputs X_7 – X_{12} have zero effect on all outputs. The oracle wraps BASELINE and passes only X_1 – X_6 to the inner evaluation function.

B VR Prompt and Output Schema

The agent receives a system prompt at the start of each run containing: (1) the oracle’s input names and bounds, (2) output names and optimization directions, (3) the constraint $Y_3 > 0.4$, and (4) a description of the hypothesize–predict–reconcile loop. No information about the underlying mechanism structure is provided. A representative excerpt:

You are optimizing a system with 6 inputs (X_1 – X_6 , each in $[0.1, 1.0]$) and 4 outputs: maximize Y_1 , minimize Y_2 , ensure $Y_3 > 0.4$, maximize Y_4 . You have a budget of 72 oracle evaluations. Before each evaluation, predict the output values. After observing results, update your causal model. Use OAT sweeps to understand input–output relationships and interaction tests to detect coupling. Your goal is to find the Pareto-optimal trade-off while building a causal understanding of the system.

At the end of each iteration, the agent’s output is parsed against a fixed schema requiring: a list of causal edges (source, target, confidence 0–1, rationale), surprise observations, new findings, and a plan for the next iteration. The full prompt text and schema definition are available in the code repository.

C Edge Detection Threshold Sensitivity

Table 4 shows edge precision and recall as a function of the detection threshold, computed across all OAT sweeps in the BASELINE and NOISY 72-evaluation runs ($n = 5$ seeds each). When a real input is varied in an OAT sweep, the output changes by 0.006 to 0.666 (depending on the edge strength and base point). When a noise input is varied, the output change is exactly 0.000. The complete separation means precision is 1.00 at all tested thresholds. We report results at 0.05 as a conservative default.

Table 4. Edge precision and recall vs. detection threshold.

Threshold	Precision	Recall	TP	FN
0.01	1.000	0.948	147	8
0.02	1.000	0.948	147	8
0.03	1.000	0.948	147	8
0.05	1.000	0.935	145	10
0.10	1.000	0.710	110	45
0.15	1.000	0.619	96	59
0.20	1.000	0.503	78	77

D Additional Diagnostics

The following metrics are available in the SynthOracle benchmark but are not central to the main paper’s claims. We report them here for completeness.

Sobol-weighted information capture. Weights each input–output edge by its Sobol total-effect index, measuring whether the agent discovers the *high-impact* edges. Across all conditions, information capture exceeds 0.96, indicating the agent consistently identifies the most important causal relationships even when raw recall is lower (e.g., Noisy at 72 evaluations: recall = 0.70, info capture = 0.996).

OAT prediction accuracy. Before each OAT sweep, the agent predicts whether each output will increase or decrease as the swept input varies. Direction accuracy—the fraction of correct increase/decrease predictions—ranges from 66% (Haiku) to 91% (Noisy Opus at 72 evaluations), against a 50% random baseline.

Sample efficiency. The number of evaluations required to reach 50%, 75%, and 90% of reference hypervolume. On BASELINE, VR requires 20–30 evaluations for 50% (vs. 10 for BO) and 50–60 evaluations for 75% (vs. 26 for BO). The gap narrows at higher thresholds, reflecting VR’s front-loaded investment in model building.

Adversarial region error. Prediction MAE in “hard” regions (near coupling boundaries, threshold transitions) versus “easy” regions. On NOISY at 144 evaluations, adversarial MAE drops 60–67% from 72 to 144 evaluations, indicating the agent progressively learns difficult regions.

E Transfer Results

We tested whether a causal model learned on BASELINE transfers to SHIFTED (same topology, altered functional forms) and REWIRED (altered topology). The agent receives the prior causal model in its initial prompt and uses a screen-first protocol: it empirically tests each claimed edge before committing to prior beliefs.

Table 5. Transfer results (Opus, 72 evaluations, $n = 3$ seeds 42–44). Effect: prior HV minus fresh HV as percentage of fresh.

Oracle	Prior HV	Fresh HV	Effect	p	Dir.
SHIFTED	$1.209 \pm .120$	$1.186 \pm .074$	+2%	0.751	2/3
REWIRED	$0.272 \pm .005$	$0.260 \pm .011$	+5%	0.246	3/3

Both comparisons are directionally positive (prior helps) but not statistically significant at $n = 3$. The SHIFTED result is particularly noisy: seed 42 shows the prior *hurting* (−10%), while seeds 43 and 44 show modest benefit. The REWIRED result is more consistent (3/3 positive) but the effect is small. We report these as directional observations requiring larger sample sizes to confirm.

F Pre-Registered Predictions

Before running the gap-closing experiments, we committed predictions for four experimental conditions. Three of four were qualitatively wrong.

Table 6. Pre-registered predictions vs. actual outcomes.

Prediction	Expected	Actual	Why we were wrong
$R^2(M \rightarrow Y)$ predicts VR/BO	Low $R^2 \rightarrow$ worse VR	Low- R^2 oracles have <i>higher</i> VR/BO	R^2 measures mechanism interpretability, not optimization difficulty; BO also struggles on low- R^2 oracles
Wrong prior hurts on SHIFTED	-9% penalty	+2% (prior helps)	Screen-first protocol filters incorrect prior edges; partial correct structure still provides value
VR crosses BO at extended budget	VR > BO at 144	VR = 99% of BO	Assumed BO was saturated at 72 evals; it was not (BO also improves with more budget)
Sonnet replicates Opus on Noisy	Same HV and strategy	HV matches; strategy differs	Correct on outcome; did not predict that Sonnet would verify noise empirically rather than dismiss by inference